

Assignment: Caches

Instructor: Elaheh Sadredini

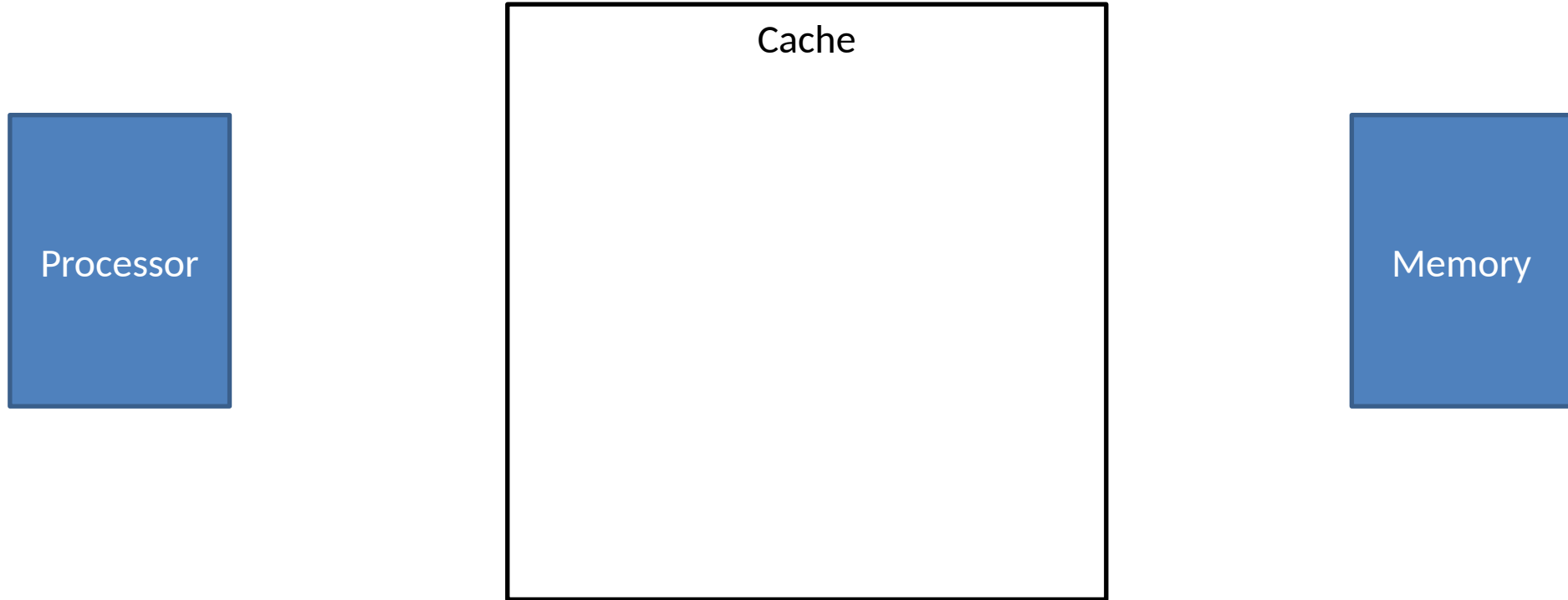
elaheh@cs.ucr.edu

University of California, Riverside

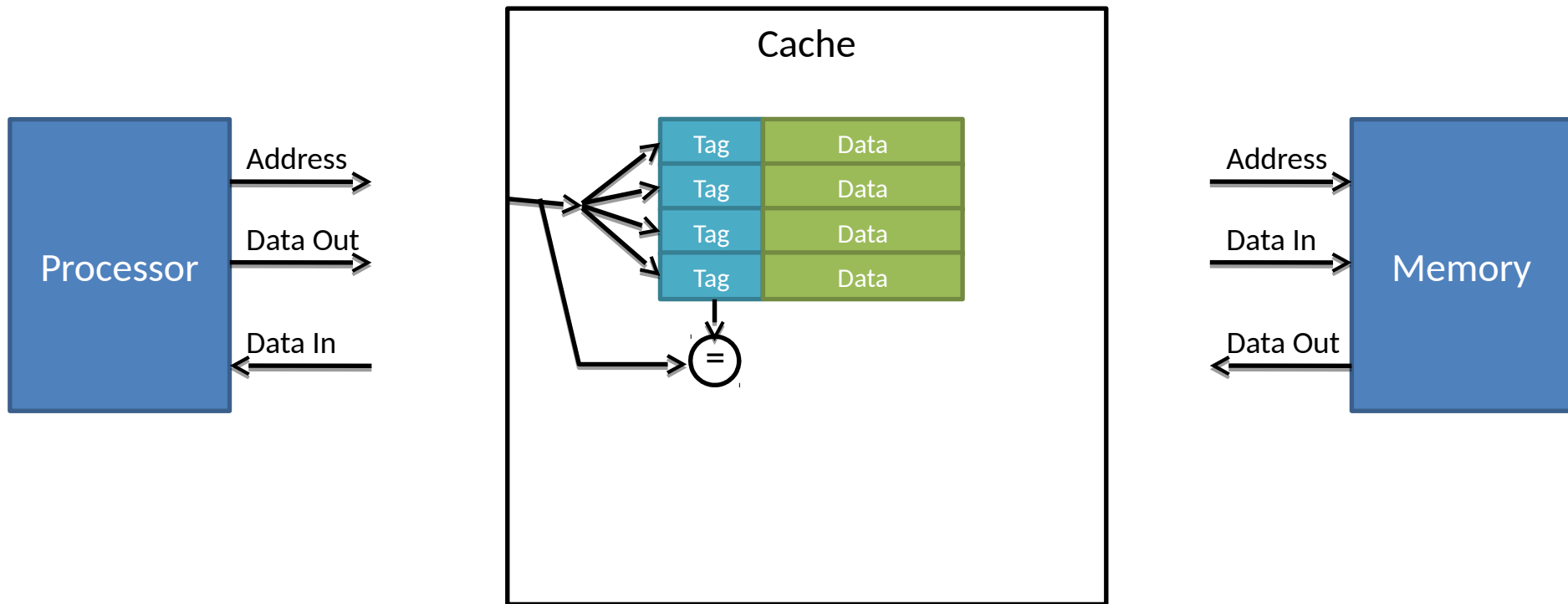
Grading (total 4 points)

- Question 1: 1 point
 - Part B: 0.4 points, Part C: 0.4 points
- Question 2: 1.2 point
 - Each part 0.4 point
- Question 3: 1.4 point
 - Part A, B, C each 0.4 point, Part D 0.2 point
- Question 4: 0.6 point
 - Each part 0.2 point

Review: What goes into and out from a cache?

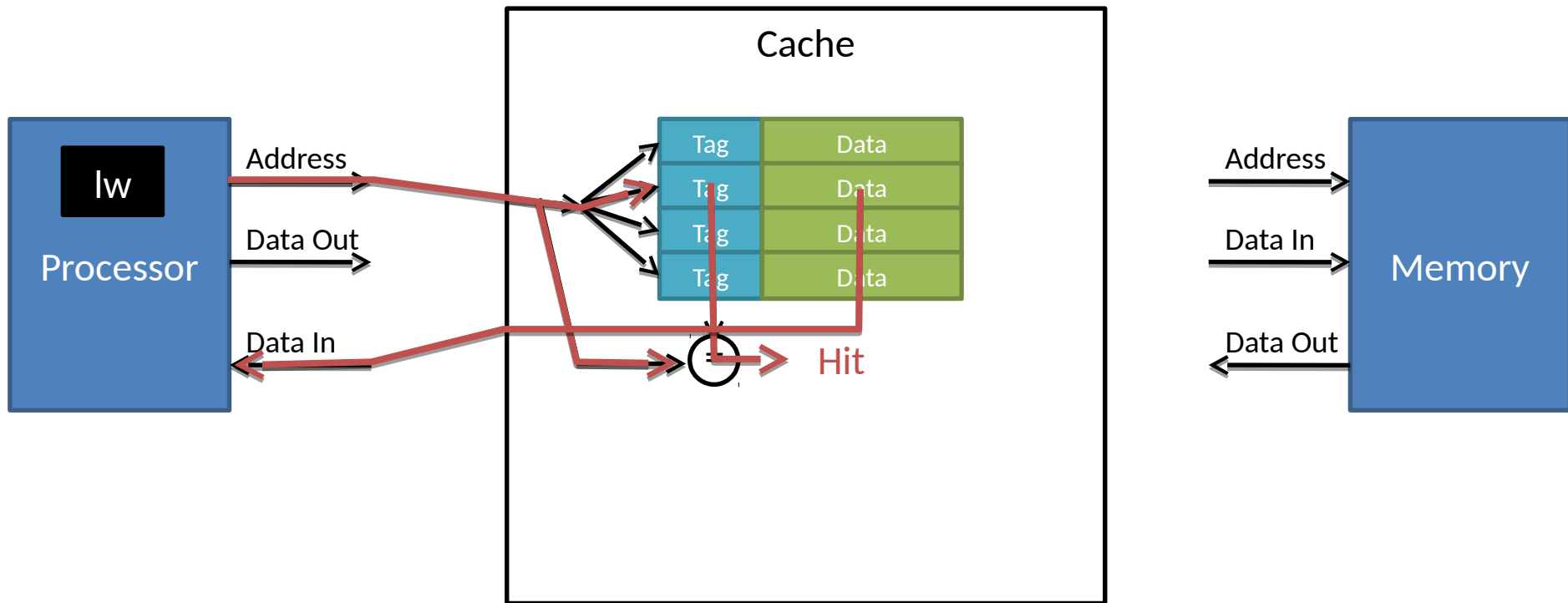


What goes into and out from a cache?



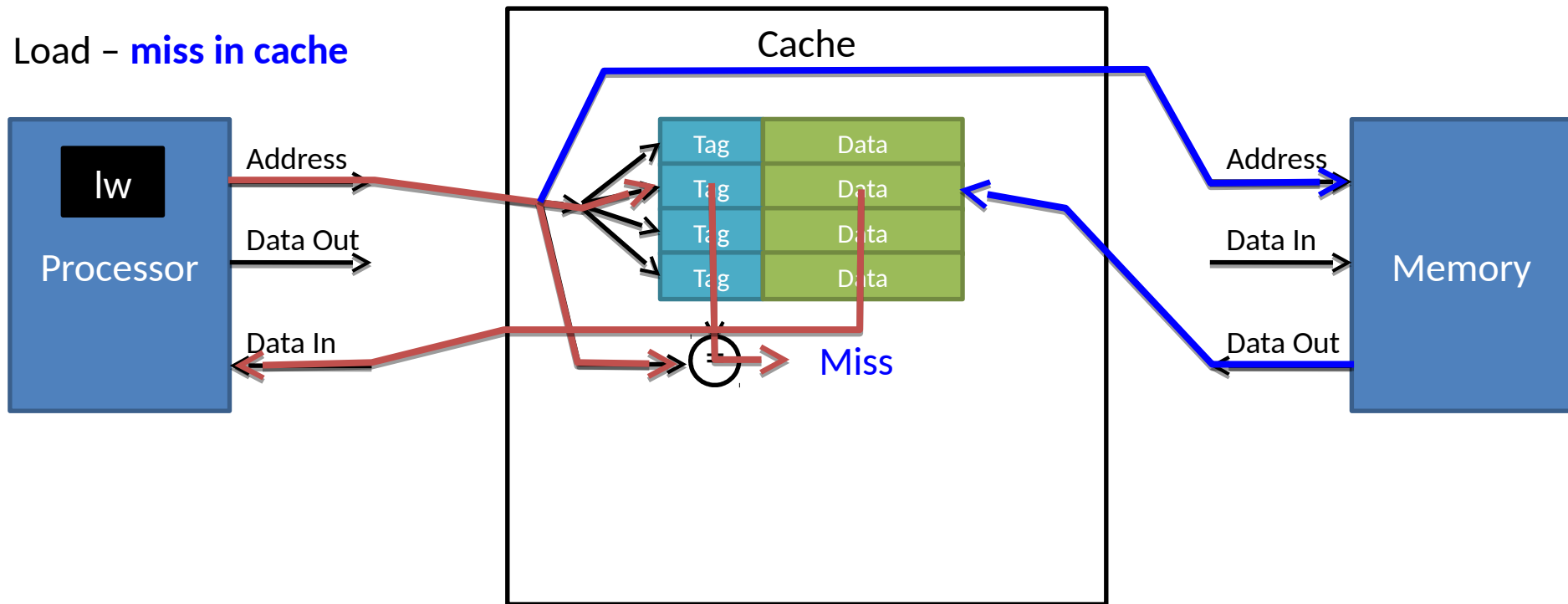
What goes into and out from a cache?

Load – **hit in cache**



What goes into and out from a cache?

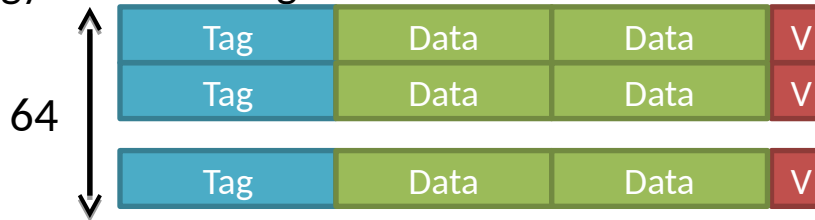
Load - **miss in cache**



Question 1: Diagram of the Caches

Draw a diagram of the following caches (note: answer for part A is provided. Follow the same structure for part B and C)

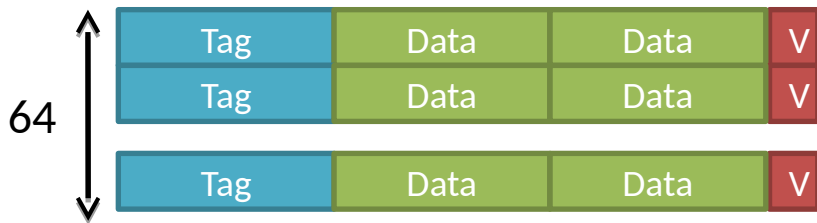
- A. Direct-mapped, cache capacity = 64 cache line, cache line size = 8-byte, word size = 4-byte, write strategy: write through



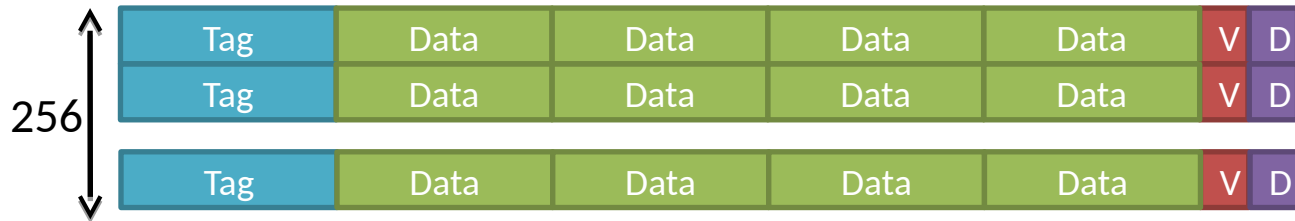
- B. Fully-associative, cache capacity = 256 cache line, cache line size = 16-byte, word size = 4-byte, write strategy: write back
- C. 4-way set-associative, cache capacity = 4096 cache line, cache line size = 64-byte, word size = 4-byte, write strategy: write back

Answer: Diagram of the Caches

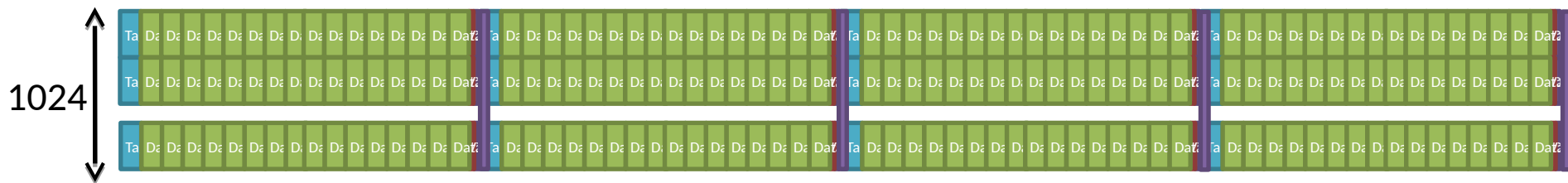
A.



B.



C.



Question 2: How are the bits used for addressing?

- Determine which bits in a 32-bit address are used for selecting the **byte (B)**, selecting the **word (W)**, indexing the **cache (I)**, and the cache **tag (T)**, for each of the following caches:
 - A. Direct-mapped, cache capacity = 64 cache line, cache line size = 8-byte, word size = 4-byte
 - B. Fully-associative, cache capacity = 256 cache line, cache line size = 16-byte, word size = 4-byte
 - C. 4-way set-associative, cache capacity = 4096 cache line, cache line size = 64-byte, word size = 4-byte

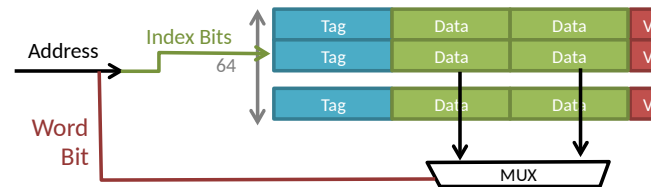
Note: cache capacity represents the maximum number of cache blocks (or cache lines) that can fit in the cache

Answer: How are the bits used for addressing?

- Determine which bits in a 32-bit address are used for selecting the **byte (B)**, selecting the **word (W)**, indexing the **cache (I)**, and the cache **tag (T)**, for each of the following caches:

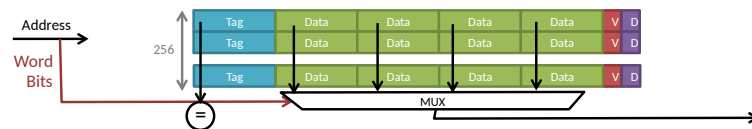
A.

- 8 byte line = 2 words per line, need 1 word bit
- 64 lines, need 6 bits for indexing
- TTTT TTTT TTTT TTTT TTTT TTTT TTTI IIII IWBB
- B=2-bit, W=1-bit, I=6-bit, T = 23-bit



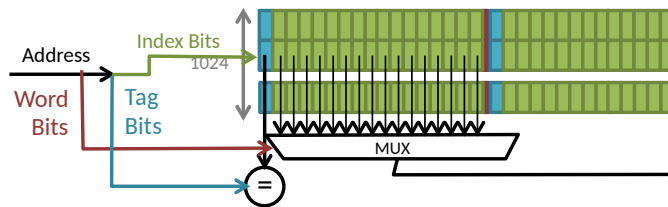
B.

- 16 byte line = 4 words per line, need 2 word bits
- 256 lines, but fully associative! 0 bits for index! (data can go anywhere)
- TTTT TTTT TTTT TTTT TTTT TTTT TTTT WWBB
- B=2-bit, W=2-bit, I=0-bit, T = 28-bit



C.

- 64 byte line = 16 words per line, need 4 word bits
- 4096 lines/4-ways = 1024 sets, need 10 bits for indexing
- TTTT TTTT TTTT TTTT IIII IIII IIWW WWBB
- B=2-bit, W=4-bit, I=10-bit, T = 16-bit



Question 3: How much overhead for the caches?

- Calculate the number of bits used for actual data and overhead (i.e., tag, valid bit, dirty bit) for each of the following caches (from Question 1)
 - A. Direct-mapped, cache capacity = 64 cache line, cache line size = 8-byte, word size = 4-byte, **write strategy: write through**
 - B. Fully-associative, cache capacity = 256 cache line, cache line size = 16-byte, word size = 4-byte, **write strategy: write back**
 - C. 4-way set-associative, cache capacity = 4096 cache line, cache line size = 64-byte, word size = 4-byte, **write strategy: write back**
 - D. How the cache line size affect the overhead?

Answer: How much overhead for the caches?

A.

- TTTT TTTT TTTT TTTT TTTT TTTI IIII IWBB
- Data: $64 \text{ lines} * (8 \text{ bytes/line}) = 512 \text{ bytes} = 4096 \text{ bits}$
- Overhead: $64 \text{ lines} * (23 \text{ bits/tag/line} + 1 \text{ valid bit/line}) = 1536 \text{ bits}$
- 73% data

B.

- TTTT TTTT TTTT TTTT TTTT TTTT TTTT WWBB
- Data: $256 \text{ lines} * (16 \text{ bytes/line}) = 4096 \text{ bytes} = 32768 \text{ bits}$
- Overhead: $256 \text{ lines} * (28 \text{ bits/tag/line} + 1 \text{ valid bit/line} + 1 \text{ dirty bit/line}) = 7680 \text{ bits}$
- 81% data

C.

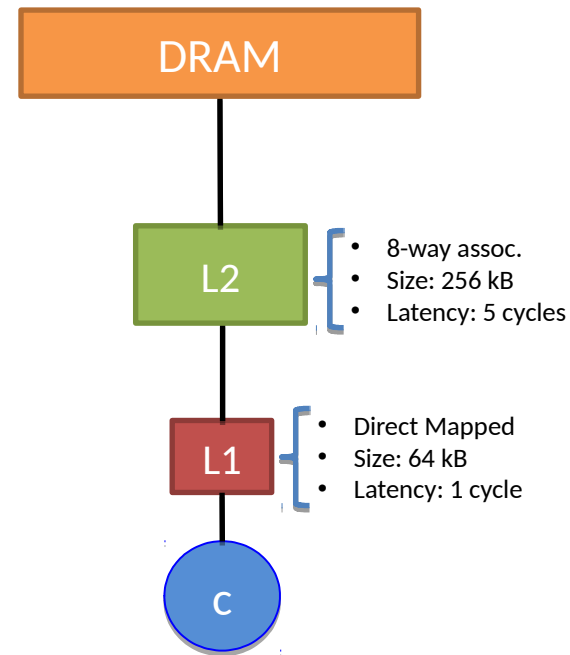
- TTTT TTTT TTTT TTTT IIII IIII IIWW WWBB
- Data: $4096 \text{ lines} * (64 \text{ bytes/line}) = 262144 \text{ bytes} = 2097152 \text{ bits}$
- Overhead: $4096 \text{ lines} * (16 \text{ bits/tag/line} + 1 \text{ valid bit/line} + 1 \text{ dirty bit/line}) = 73728 \text{ bits}$
- 97% data

D. Larger cache lines reduce overhead! (More data bits per tag bit.)

Question 4: Average Memory Access Time

Consider the processor (on right) with two-level cache hierarchy. Suppose that **65%** of *benchmark A*'s instructions are accessed from the **L1 cache**, whereas the rest come from the **L2 cache**.

- What is the average access time for memory instructions in the benchmark
- If we change the L1 cache to be 4-way set associative we increase its flexibility and find that 85% of the instructions are now accessed from the L1. What is the average access time for memory instructions in the benchmark now?
- What is the speedup for changing the L1?



Answer: Average Memory Access Time

A.

$$- 0.65 * 1 \text{ cycle} + 0.35 * (5 \text{ cycles} + 1 \text{ cycle}) = 2.75 \text{ cycles}$$

B.

$$- 0.85 * 1 \text{ cycle} + 0.15 * (5 \text{ cycles} + 1 \text{ cycle}) = 1.75 \text{ cycles}$$

C.

- $2.75 / 1.75 = 1.57\text{X}$ faster
- Or $(2.75 - 1.75) / 2.75 = 36\%$ faster

